МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

Факультет информационных технологий

Кафедра «Инфокогнитивные технологии»

КУРСОВОЙ ПРОЕКТ

на тему: «*Разработка системы мониторинга и бронирования ресурсов студенческого общежития №6 Московского Политеха.*»

Направление подготовки 09.03.03 «Прикладная информатика»

Профиль «Разработка и интеграция бизнес-приложений»

Выполнила:

Андрианова Мария Ивановна

Москва 2026

Введение

В условиях студенческого быта Московского Политеха организация доступа к общим ресурсам, в частности к оборудованию прачечных комнат, представляет собой значительную практическую проблему. В общежитии №6, где проживает более 1200 человек, на первом этаже расположено всего 5 стиральных и 3 сушильные машины. В отсутствие системы управления процесс стирки превращается в хаотичное ожидание, что приводит к нерациональной трате личного времени студентов, возникновению конфликтных ситуаций и неэффективному использованию инфраструктуры.

Проблематика заключается в системном отсутствии координации доступа к оборудованию прачечной. Студенты не имеют возможности удаленно проверить занятость машин или заранее зарезервировать временной слот, вынуждены физически находиться в помещении для ожидания своей очереди. Ситуацию усугубляют сопутствующие организационные проблемы: привычное несвоевременное извлечение белья пользователями, что приводит к длительным простоям работающего оборудования, отсутствие централизованной информации о действующих способах оплаты и оперативного оповещения о неисправностях машин. Совокупность этих факторов снижает общую доступность сервиса.

На сегодняшний день эффективные решения данной проблемы отсутствуют. Единственный инструмент координации — общий чат жильцов общежития в мессенджере — не решает системных проблем, а лишь констатирует их. Чат превращается в поток сообщений с просьбами забрать вещи из машин, не предлагая механизмов планирования и предотвращения конфликтных ситуаций. Таким образом, фактически отсутствует какая-либо система управления доступом к ресурсам прачечной, что подтверждает актуальность разработки специализированного решения.

В качестве решения предлагается разработать специализированный веб-сервис «СтирайБезОчереди». Система представляет собой клиент-серверное веб-приложение на технологическом стеке PHP и MySQL, предназначенное для трансформации хаотичного процесса в управляемый сервис. Ключевой функционал включает: онлайн-бронирование слотов с визуализацией актуального статуса всех машин в реальном времени; встроенную систему рейтинга пользователей, стимулирующую соблюдение временных рамок;

механизмы информирования о способах оплаты и оперативного сообщения о неисправностях; а также административную панель для мониторинга и аналитики.

1 Постановка задачи

Цель работы — разработать специализированный веб-сервис «СтирайБезОчереди», представляющий собой клиент-серверное веб-приложение на базе PHP и MySQL, управляемый и справедливый доступ к ресурсам прачечной студенческого общежития №6 Московского Политеха.

Задачи:

1. Анализ предметной области и проектирование базы данных: выявление ключевых сущностей и процессов, разработка инфологической и даталогической моделей данных, создание физической модели в СУБД MySQL [1].

2. Проектирование и реализация архитектуры веб-приложения: создание модульной структуры по принципу разделения ответственности, организация взаимодействия между клиентской частью (HTML, CSS, JavaScript) и серверной логикой на PHP, обеспечение безопасности данных.

3. Разработка модуля аутентификации и авторизации: реализация механизмов регистрации и входа в систему, внедрение ролевой модели с разграничением прав доступа к функциям системы.

4. Реализация ядра системы — модуля бронирования: создание многошагового интерфейса для выбора даты, времени и конкретной машины, разработка алгоритма проверки доступности слотов и предотвращения конфликтов бронирования.

5. Разработка системы мониторинга и обратной связи: создание главной страницы с визуализацией реального статуса всех машин, реализация функционала для оперативного сообщения пользователями о неисправностях оборудования.

6. Внедрение социального механизма — рейтинговой системы: разработка интерфейса для подтверждения пунктуальности предыдущего пользователя, связь рейтинга с привилегиями при бронировании.

7. Создание административного интерфейса: разработка панели

управления для администратора, включающей функции ручного изменения статусов машин, обработки сообщений о поломках, просмотра расширенной истории бронирований и визуальной аналитики загруженности.

8. Тестирование, отладка и развертывание: проверка корректности работы всех модулей системы, взаимодействия пользователей, целостности данных и безопасности.

Выбранные средства разработки:

- PHP — для реализации бизнес-логики приложения [2].
- MySQL — для хранения информации.
- HTML5, CSS3 — для создания пользовательского интерфейса.
- JavaScript — для реализации интерактивных элементов.
- Sprinthost — хостинг для развертывания веб-приложения [3].
- Git — для ведения истории разработки.

2 Проектирование и разработка

Приложение реализует клиент-серверную архитектуру и предоставляет функциональные возможности в соответствии с ролевой моделью доступа. Пользователь может зарегистрироваться, указав данные студента (номер студенческого билета, номер комнаты, ФИО, email и пароль), а затем пройти авторизацию.

Неавторизованный пользователь (Гость) имеет возможность просматривать главную страницу с актуальным статусом всех машин в прачечной. Он может зарегистрироваться в системе как студент общежития №6 или войти в систему с помощью email и пароля.

Студент (основной пользователь) имеет возможность просматривать свои личные данные в профиле. Он может просматривать полную историю своих бронирований и отменять свои брони. Студент осуществляет многошаговое бронирование машины через последовательный выбор даты с ограничениями, зависящими от рейтинга, выбора времени начала стирки или сушки и выбора конкретной свободной машины на интерактивной карте прачечной. Студент может подтверждать пунктуальность предыдущего пользователя, отмечая забрал

ли он вещи вовремя, а также сообщать о неисправностях оборудования, что автоматически переводит машину в статус на ремонте.

Администратор может управлять статусами машин вручную переводя их в состояния: свободно, занято или на ремонте. Администратор просматривает и обрабатывает все сообщения о поломках оборудования, просматривает полную историю бронирований всех пользователей с расширенной фильтрацией по дате, времени, пользователю и машине. Он анализирует статистику использования прачечной через визуальные диаграммы.

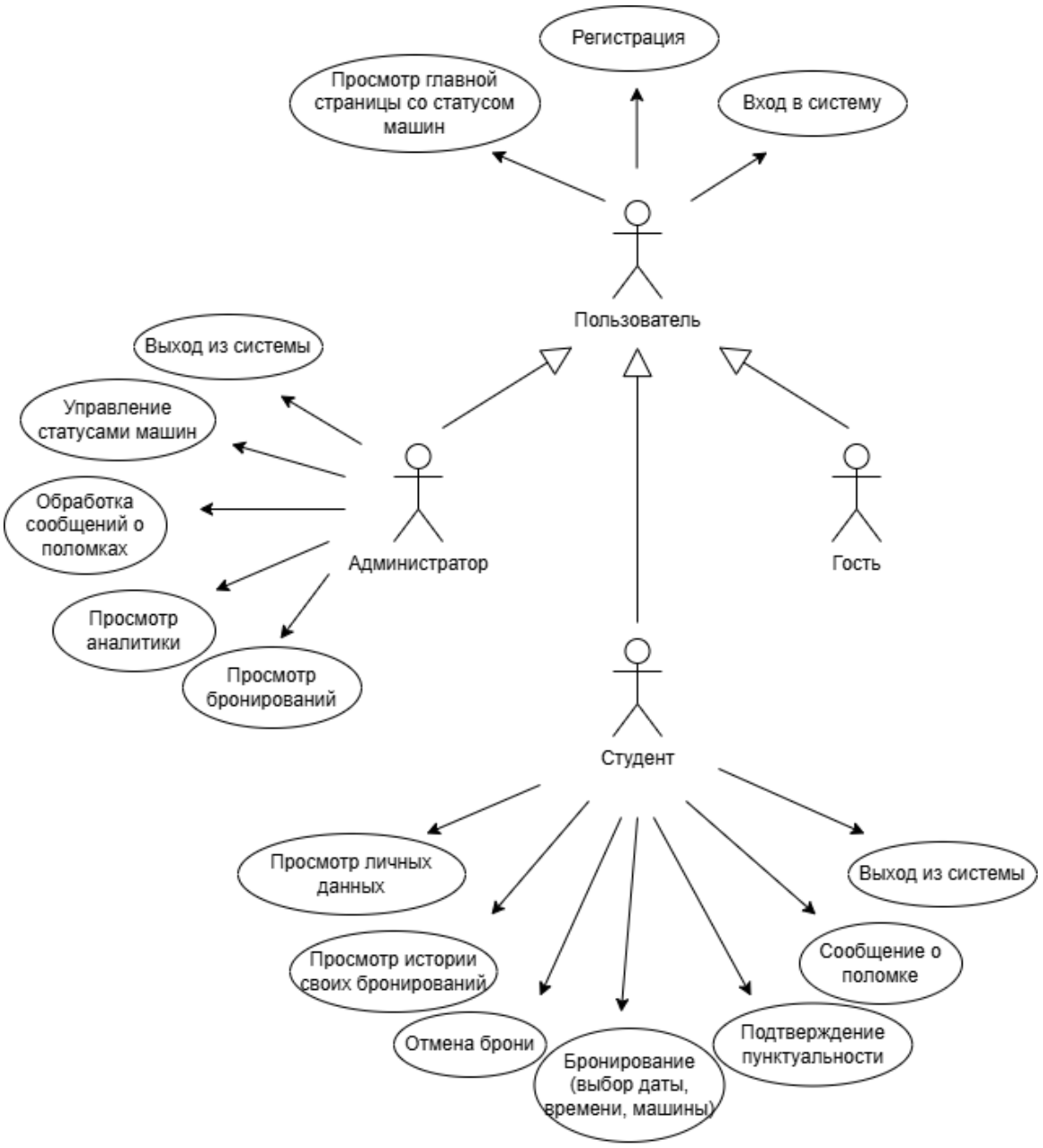


Рисунок 1 – Диаграмма вариантов использования

На рисунке 1 представлена диаграмма вариантов использования

приложения, отражающая основные сценарии взаимодействия пользователей с системой.

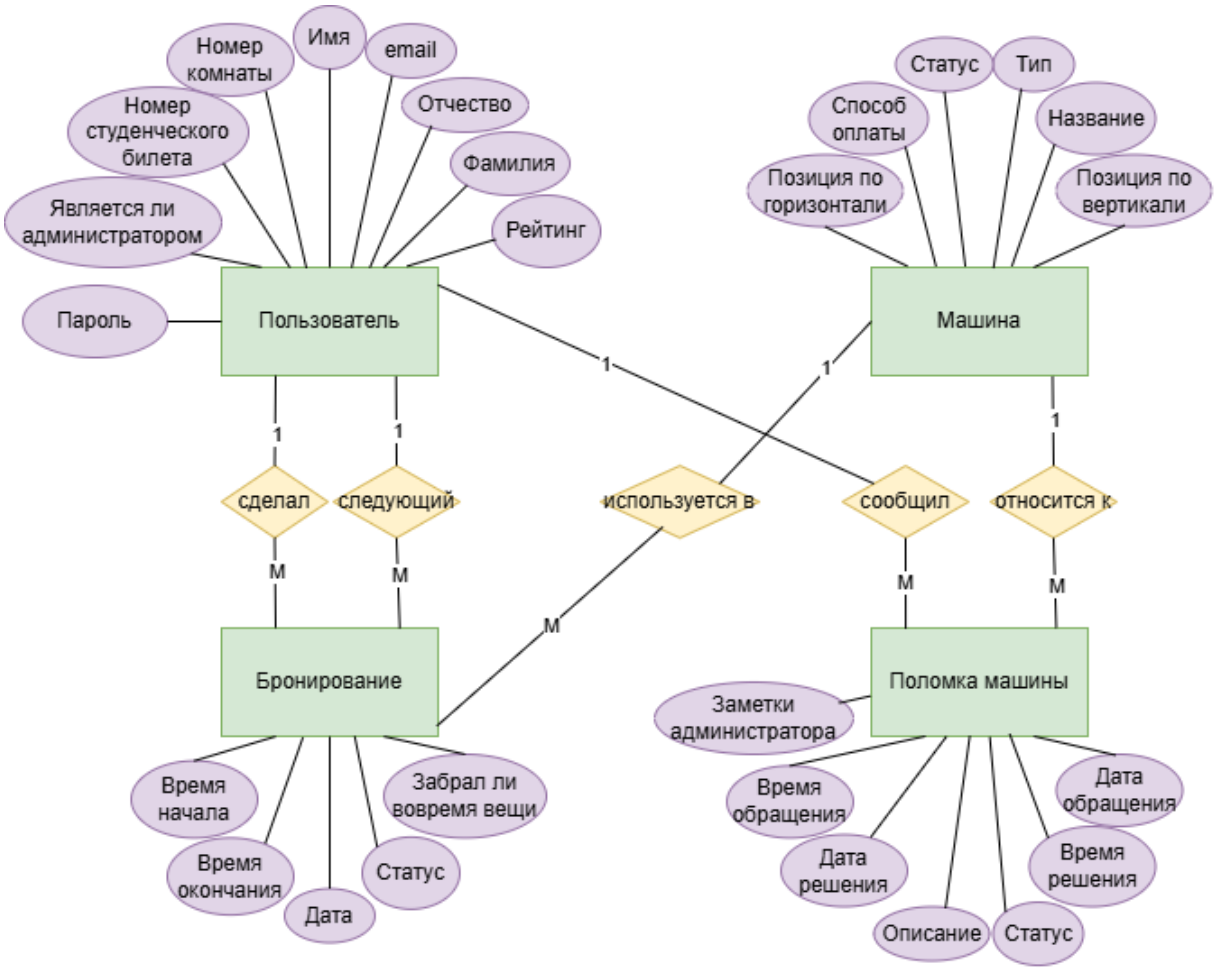


Рисунок 2 – Инфологическая модель базы данных

На рисунке 2 представлена инфологическая модель базы данных системы, которая отражает предметную область управления доступом к ресурсам прачечной студенческого общежития.

Множества сущностей включают Пользователя, который является физическим лицом, проживающим в общежитии и имеющим доступ к услугам прачечной (студент или администратор). Машина представляет собой оборудование для стирки или сушки белья в прачечной общежития. Бронирование — это временной слот для использования конкретной машины определенным пользователем. Поломка машины — это сообщение о неисправности или проблеме с конкретной машиной.

Связи между сущностями установлены следующие пять связей:

Связь Пользователь → Бронирование (1:M). Каждый пользователь может сделать несколько бронирований, но каждое бронирование сделано только одним

пользователем.

Связь Бронирование → Пользователь (M:1). Каждое бронирование имеет следующего пользователя, который подтверждает пунктуальность, при этом каждый пользователь может быть следующим для нескольких бронирований.

Связь Пользователь → Поломка машины (1:M). Каждый пользователь может сообщить несколько поломок, но каждая поломка сообщена только одним пользователем.

Связь Машина → Бронирование (1:M). Каждая машина используется в нескольких бронированиях, но каждое бронирование использует только одну машину.

Связь Машина → Поломка машины (1:M). Каждая машина может иметь несколько поломок в своей истории, но каждая поломка относится только к одной машине.

На рисунке 3 представлена даталогическая модель базы данных системы. Модель включает четыре таблицы: Пользователь (E1) с первичным ключом id и альтернативными ключами Номер_студенческого_билета (AK1) и Email (AK2), Машина (E2) с первичным ключом id и альтернативными ключами Название (AK1) и составным ключом Позиция_по_вертикали + Позиция_по_горизонтали (AK2), Бронирование (E3) с первичным ключом id и составными альтернативными ключами id_машины + Дата + Время_начала (AK1) и id_машины + Дата + Время_окончания (AK2), а также Поломка машины (E4) с первичным ключом id и составным альтернативным ключом id_машины + Дата_обращения + Время_обращения (AK).

Все связи между таблицами в данной модели являются определёнными неидентифицирующими обязательными. Связи определённые, потому что каждая связь имеет тип "один ко многим где родительская сущность со стороны "один"а дочерняя со стороны "много". Связи неидентифицирующие потому что внешние ключи родительских сущностей не входят в состав первичных ключей дочерних сущностей. Большинство связей обязательные, потому что дочерние сущности зависимы по существованию от родительских за исключением связи Бронирование → Пользователь через атрибут id_следующего которая является необязательной.

Внешние ключи реализуют следующие связи: FK1 и FK3 в Бронировании ссылаются на Пользователя, FK2 в Бронировании ссылается на Машину, FK1

в Поломке машины ссылается на Машину, FK2 в Поломке машины ссылается на Пользователя. Система внешних ключей обеспечивает целостность данных с использованием каскадных операций обновления и ограничивающих операций удаления что гарантирует согласованность данных во всей системе.

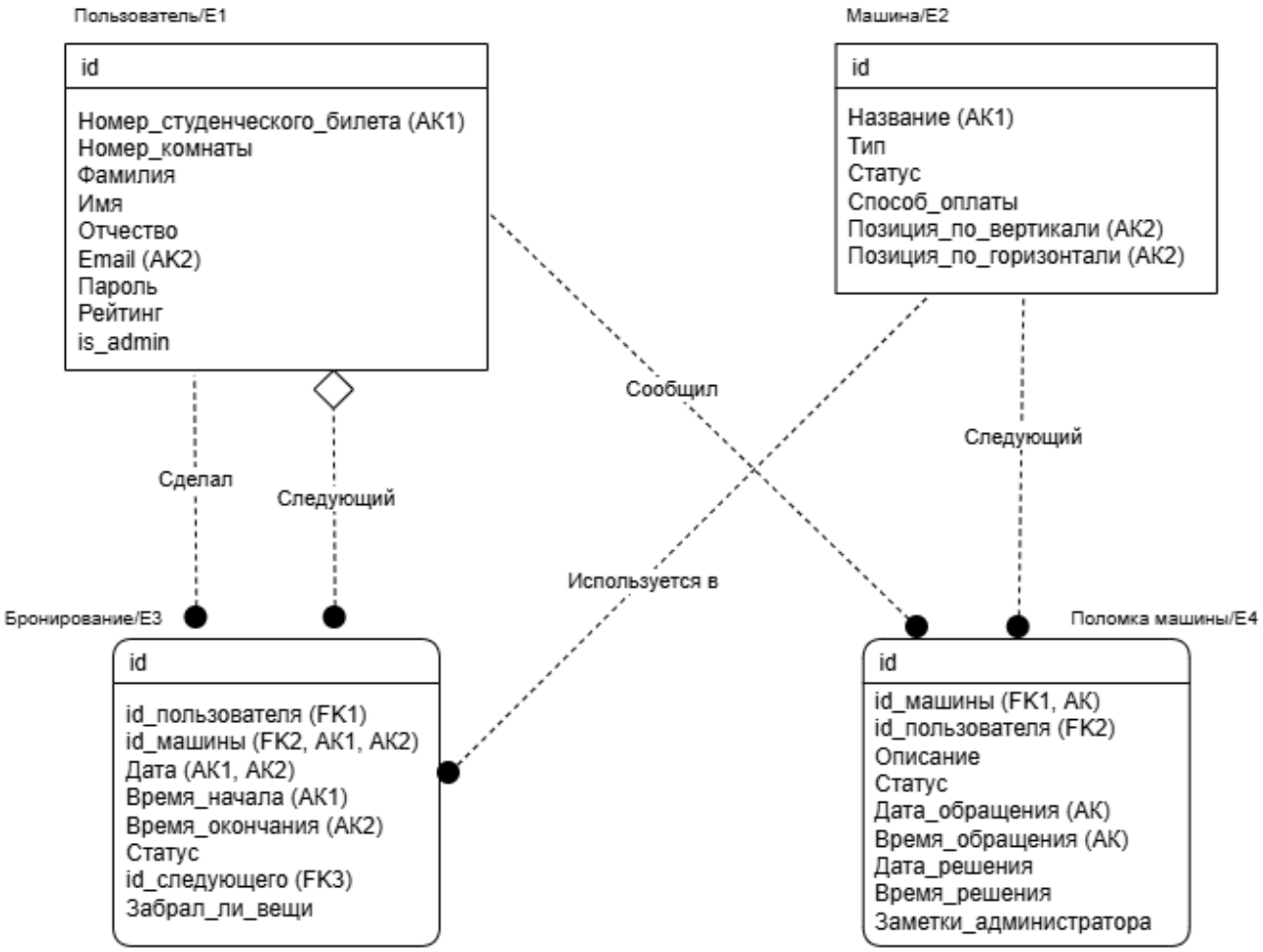


Рисунок 3 – Даталогическая модель базы данных

На основе этой модели разработана физическая модель, SQL-скрипт которой можно посмотреть в приложении А.

Приложение «СтирайБезОчереди» сделано по модульному принципу, каждая часть системы отвечает за свою задачу. Все модули работают вместе через общую базу данных. Сайт работает по схеме клиент-сервер. В браузере у пользователя отображается интерфейс на HTML и CSS, а на сервере выполняется код на PHP, который работает с базой данных MySQL.

Главная страница — это центр сайта. От неё идут все остальные разделы. Вверху есть меню, которое меняется в зависимости от того, кто зашёл — студент или администратор. Модуль входа и регистрации отвечает за создание аккаунтов и вход в систему. Он проверяет, кто зашёл, и даёт нужные права: обычному

студенту — одни возможности, администратору — другие. Модуль бронирования — это несколько шагов подряд. Сначала выбираешь дату, потом время, потом конкретную машину на карте. Система сама проверяет, свободна ли машина в это время, и не даёт выбрать занятый слот. Модуль карты прачечной показывает всё оборудование в реальном времени. Каждая машина — это цветной квадратик: зелёный — свободна, красный — занята, серый — в ремонте. Тут же видно, сколько времени осталось до конца стирки и как платить. Модуль пунктуальности отвечает за рейтинг студентов. Если кто-то забирает вещи не вовремя — падает. Чем выше рейтинг, тем на больше дней вперёд можно бронировать. Модуль для администратора позволяет управлять машинами вручную, смотреть все бронирования, обрабатывать жалобы на поломки и изучать статистику — когда прачечная больше всего занята.

Сам сайт устроен просто. Главная страница — карта прачечной. Для студентов есть кнопки: «Личный кабинет», «Забронировать», «Сообщить о поломке», «Подтвердить пунктуальность». Для администратора — «Управление машинами», «История бронирований», «Статистика». Интерфейс сайта простой и удобный. Цвета в дизайне помогают быстро понять что к чему. Адаптивный дизайн, обеспечивающий корректное отображение на мобильных устройствах. Элементы интерфейса масштабируются в зависимости от размера экрана.

3 Реализация приложения

Кодовая база приложения организована в виде набора PHP-скриптов, непосредственно реализующих функциональность, описанную в диаграмме вариантов использования. Центральным механизмом является подготовленный SQL-запрос с проверкой временных конфликтов при бронировании, который обеспечивает корректную работу системы при одновременном доступе нескольких пользователей.

Ядро системы — проверка доступности слота реализовано в book_machine.php. Этот важный запрос предотвращает двойное бронирование одной машины на одно время, обеспечивая целостность данных, заложенную в даталогической модели через альтернативные ключи AK1 и AK2. Механизм проверки исключает возможность пересечения временных интервалов для каждого оборудования. (см. листинг 1).

Листинг 1 - Проверка конфликтов бронирования в book_machine.php

```
$stmt = $pdo->prepare("
    SELECT id
    FROM bookings
    WHERE id_machine = ?
    AND date = ?
    AND status != 'отменено'
    AND start_time < ?
    AND end_time > ?
");
$stmt->execute([$machine_id, $date, $end_time, $start_time]);
$conflicting_booking = $stmt->fetch();
```

Система репутации с автоматическим штрафом реализована в ‘confirm_punctuality.php‘. При подтверждении непунктуальности предыдущего пользователя система автоматически снижает его рейтинг на 5 баллов, что напрямую влияет на его привилегии бронирования. Данный механизм реализует принцип социальной ответственности, побуждая пользователей своевременно освобождать оборудование для следующих жильцов общежития (см. листинг 2).

Листинг 2 - Обновление рейтинга пользователя в confirm_punctuality.php

```
if ($was_on_time == 0) {
    $new_rating = max(0, $booking['previous_user_rating'] - 5);
    $stmt = $pdo->prepare("
        UPDATE users
        SET rating = ?
        WHERE id = ?
    ");
    $stmt->execute([$new_rating, $booking['id_user']]);
}
```

Автоматическое обновление статусов машин в реальном времени — ключевая функция системы, реализованная в update_machine_status.php. Скрипт автоматически переводит машины между статусами «занято» и «свободно» по истечении времени бронирования, обеспечивая актуальность информации на главной странице системы. Данный механизм выполняется при каждом обращении пользователя к интерфейсу, что гарантирует отображение точного состояния оборудования без необходимости ручного вмешательства администратора или пользователей (см. листинг 3).

Листинг 3 - Автоматическое обновление статусов машин в update_machine_status.php

```
$stmt = $pdo->prepare("
    SELECT *
    FROM bookings
    WHERE id_machine = ?
    AND date = ?
    AND status = 'забронировано'
    AND start_time <= ?
    AND end_time > ?
");
$stmt->execute([$machine['id'], $current_date,
$current_time, $current_time]);
$active_booking = $stmt->fetch();

if ($active_booking && $machine['status'] != 'занято') {
    $stmt = $pdo->prepare("UPDATE machines
                           SET status = 'занято'
                           WHERE id = ?");
    $stmt->execute([$machine['id']]);
}
```

Динамическое ограничение бронирования по рейтингу — бизнес-логика, реализующая систему поощрений. Функция в select_date.php определяет, на сколько дней вперёд может бронировать пользователь, основываясь на его текущем рейтинге (см. листинг 4).

Листинг 4 - Определение лимита бронирования по рейтингу в select_date.php

```
function getMaxBookingDays($rating) {
    if ($rating >= 80) return 7;
    elseif ($rating >= 60) return 5;
    elseif ($rating >= 40) return 3;
    elseif ($rating >= 20) return 1;
    else return 0;
}
```

Ролевая модель доступа с проверкой прав администратора реализована через проверку сессии. Все административные функции начинаются с этого кода в manage_machines.php, который предотвращает несанкционированный доступ к управляющему функционалу. Проверка выполняется на уровне каждого административного скрипта, гарантируя защиту управляющих операций от выполнения другими лицами (см. листинг 5).

Листинг 5 - Проверка прав администратора в manage_machines.php

```
if (!isset($_SESSION['user_id']) || !$_SESSION['is_admin']) {
    header("Location: index.php");
    exit();
}
```

Сбор и анализ статистики реализован в файле statistics.php. Модуль анализирует исторические данные о бронированиях для выявления закономерностей использования прачечной. Статистика группируется по дням недели и часам, что позволяет определить периоды наибольшей загруженности оборудования (см. листинг 6).

Листинг 6 - Сбор статистики бронирований в statistics.php

```
$stmt = $pdo->prepare("
    SELECT
        DAYOFWEEK(date) as day_of_week,
        HOUR(start_time) as hour,
        COUNT(*) as count
    FROM bookings
    GROUP BY DAYOFWEEK(date), HOUR(start_time)
    ORDER BY day_of_week, hour
");
```

Визуализация данных выполнена в виде heatmap-диаграммы, где интенсивность цвета соответствует количеству бронирований. Для каждого часа рассчитывается процентное соотношение относительно общего числа бронирований в конкретный день недели, обеспечивая наглядное представление распределения нагрузки (см. листинг 7).

Листинг 7 - Расчет визуальных параметров heatmap-диаграммы в statistics.php

```
$percentage = $dayTotal > 0 ? round(($count / $dayTotal) * 100, 1) : 0;
$height = $maxCount > 0 ? max(5, ($count / $maxCount) * 120) : 5;
```

Безопасная обработка аутентификации с хэшированием паролей реализована в login.php. Система использует password_verify() для безопасной проверки паролей, что соответствует современным требованиям информационной безопасности. Данный подход защищает учётные записи пользователей от несанкционированного доступа. (см. листинг 8).

Листинг 8 - Аутентификация пользователя в login.php

```
if ($user) {
    if (password_verify($password, $user['password'])) {
        $_SESSION['user_id'] = $user['id'];
        $_SESSION['is_admin'] = $user['is_admin'];
        header("Location: index.php");
        exit();
    }
}
```

Каскадное обновление статусов при отмене брони — важный аспект целостности данных. При отмене брони в profile.php система проверяет, остались ли другие активные брони на эту машину (см. листинг 9).

Листинг 9 - Обработка отмены бронирования в profile.php

```
if (!$other_bookings) {
    $stmt = $pdo->prepare("
        UPDATE machines
        SET status = 'свободно'
        WHERE id = ?
    ");
    $stmt->execute([$booking['id_machine']]);
}
```

Каждый из этих листингов демонстрирует ключевые аспекты реализации: безопасность данных через подготовленные запросы, бизнес-логику системы репутации, автоматическое управление состояниями и ролевой контроль доступа. Вся функциональность напрямую соответствует требованиям, описанным в предыдущих разделах проекта.

4 Основные сценарии работы приложения

Демонстрация возможностей приложения, к которым относятся авторизация пользователей, регистрация нового аккаунта, возможности студента и возможности администратора.

Авторизация пользователей происходит на странице входа (рисунок 4), которая открывается при переходе по ссылке "Войти"и предлагает ввести email и пароль. При правильно введённых данных система создаёт сессию пользователя и перенаправляет на главную страницу с интерактивной картой прачечной. В противном случае система выводит сообщение об ошибке с красным фоном, информирующее о неверных учётных данных (рисунок 5).

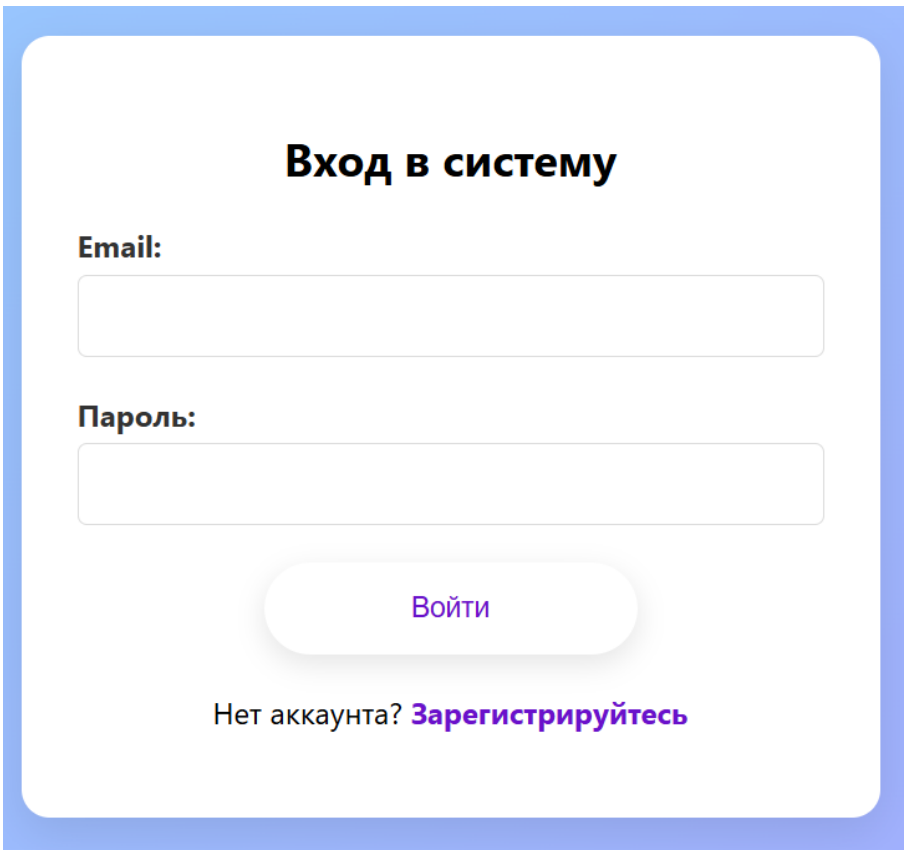


Рисунок 4 – Авторизация пользователя

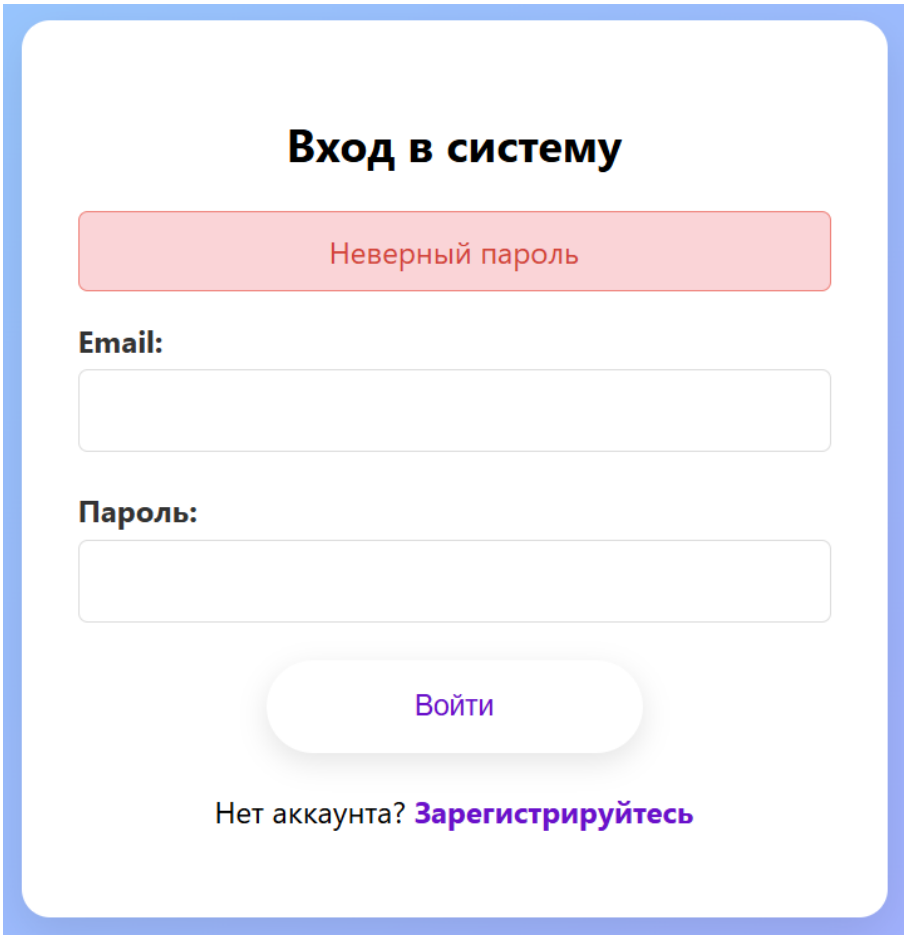


Рисунок 5 – Ошибка ввода данных

Регистрация нового аккаунта происходит при нажатии на кнопку «Зарегистрироваться». Пользователю предлагается заполнить форму (рисунок 6) с обязательными полями: номер студенческого билета, номер комнаты, фамилия, имя, email и пароль. При нажатии на кнопку «Зарегистрироваться» система проверяет уникальность номера студенческого билета и email. Если данные уже существуют в системе, выводится сообщение об ошибке (рисунок 4).

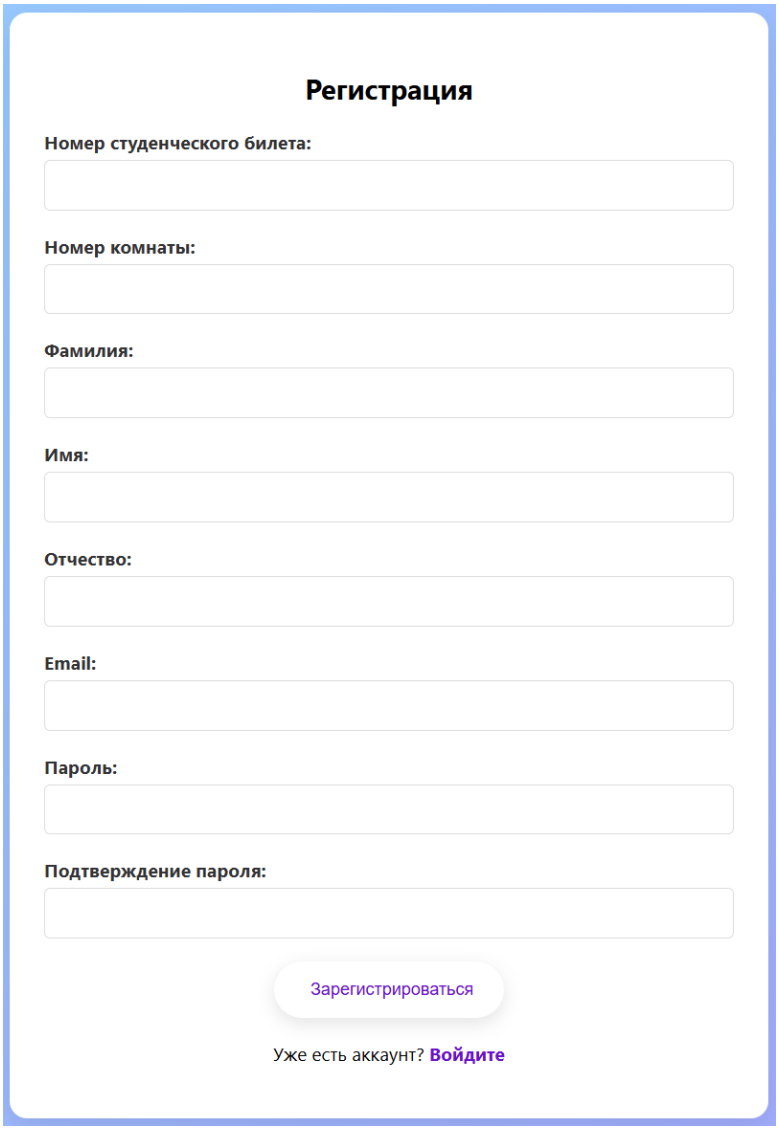


Рисунок 6 – Форма регистрации

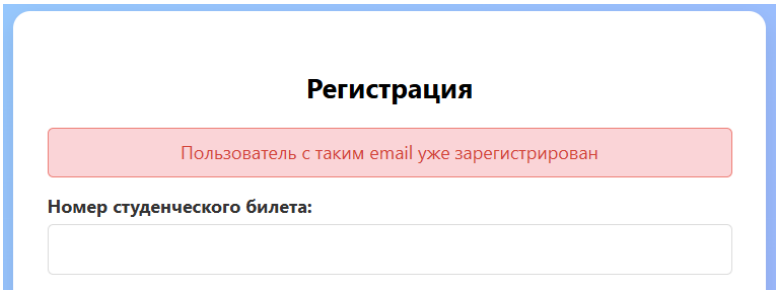


Рисунок 7 – Проверка на уникальность данных

После авторизации студент видит главную страницу с интерактивной картой прачечной (рисунок 8), где каждая машина представлена цветным блоком: зелёный — свободна, красный — занята, серый — на ремонте. Для занятых машин отображается обратный отсчёт времени до освобождения. Данное расположение карточек соответствует фактическому расположению машин в реальной жизни (см. приложение Б).

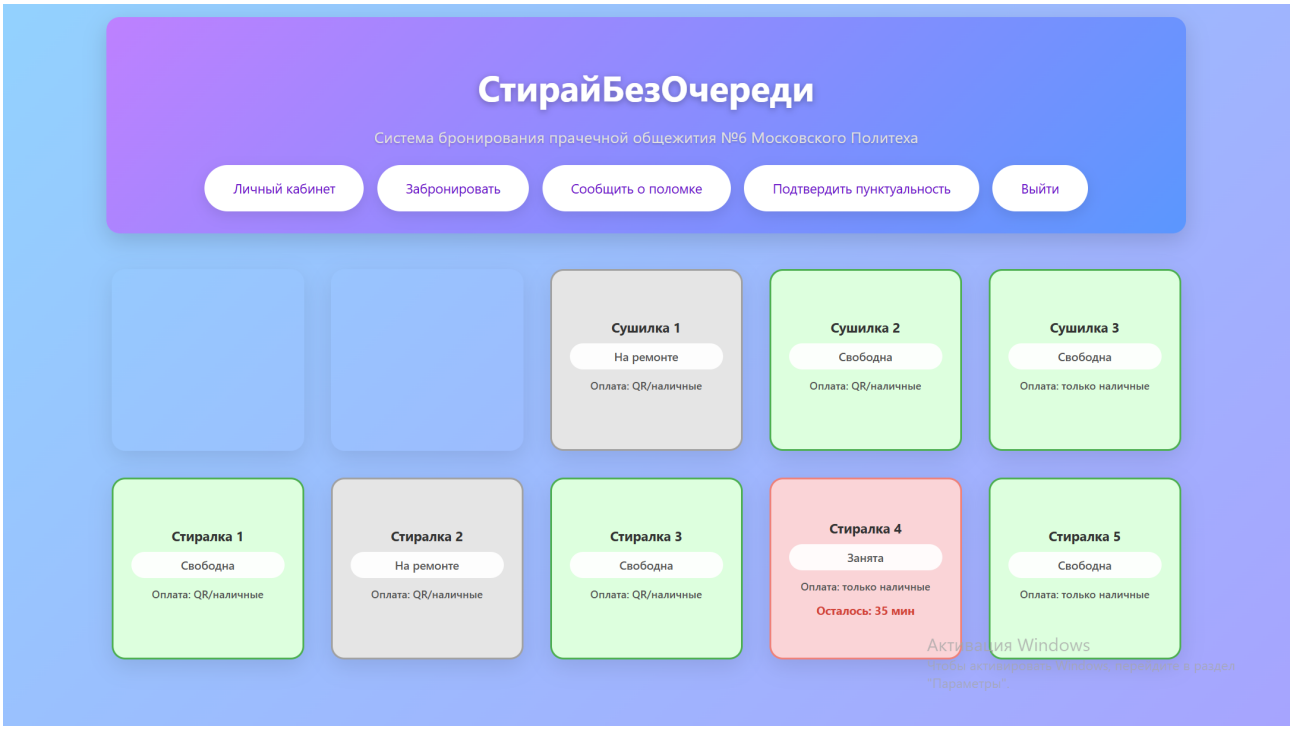


Рисунок 8 – Главная страницас картой прачечной

Для бронирования машины студент нажимает кнопку «Забронировать» и проходит трёхэтапный процесс. Сначала он выбирает дату (рисунок 9) — система показывает только те дни, которые доступны для его рейтинга. Затем выбирает время (рисунок 10) из списка свободных часовых слотов. В конце выбирает конкретную машину на интерактивной карте прачечной, где свободное оборудование подсвечено зелёным цветом. Если на каком-то этапе студент ошибётся или выберет уже занятый вариант, система сразу подскажет, что нужно исправить. Весь процесс занимает несколько минут и не требует специальных знаний (рисунок 11).

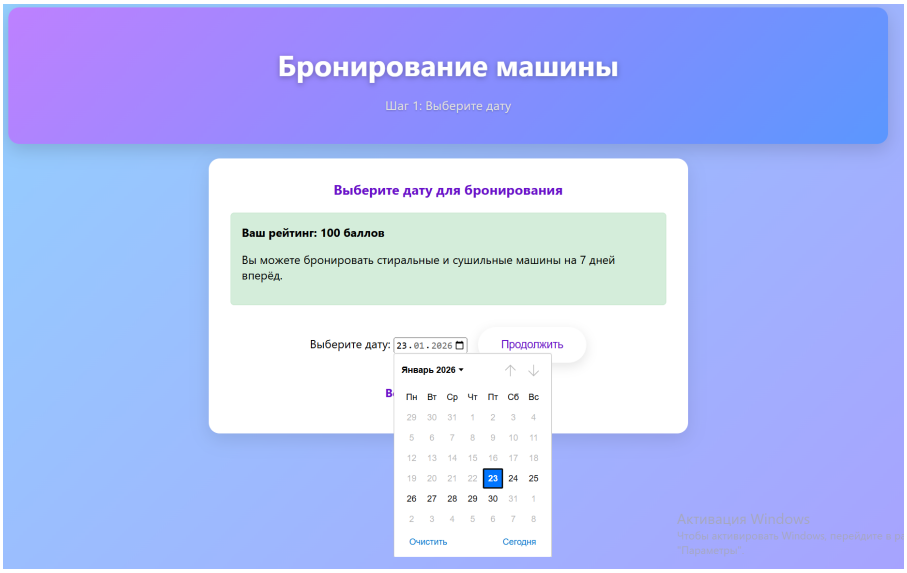


Рисунок 9 – Выбор даты

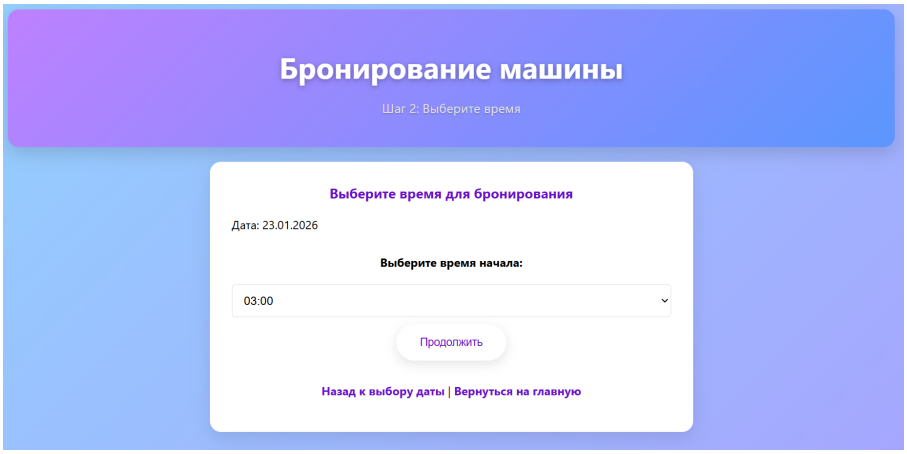


Рисунок 10 – Выбор времени

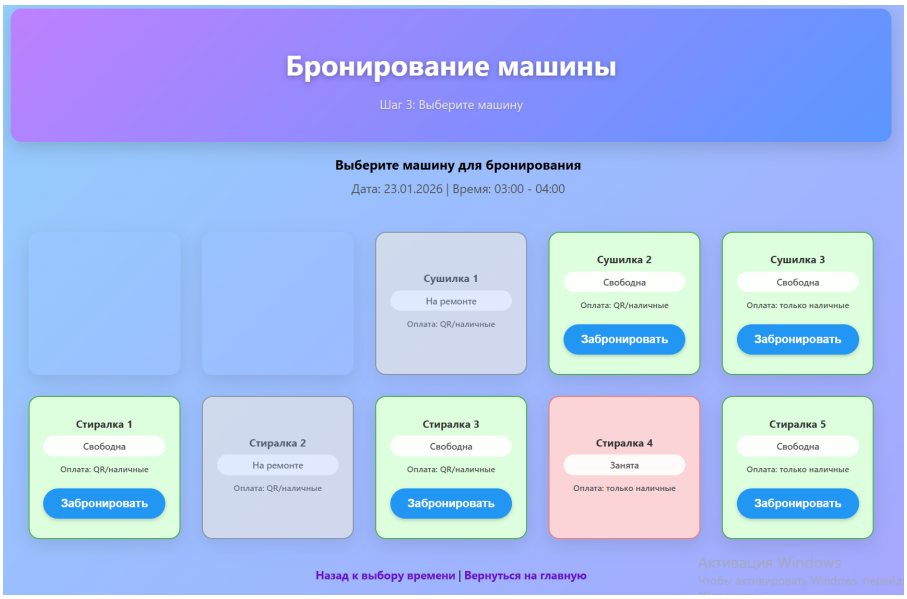


Рисунок 11 – Выбор машины

После подтверждения бронирования (рисунок 12), появляется сообщение об успешном исходе (рисунок 13). В личном кабинете студент может просматривать историю своих бронирований (рисунок 14).

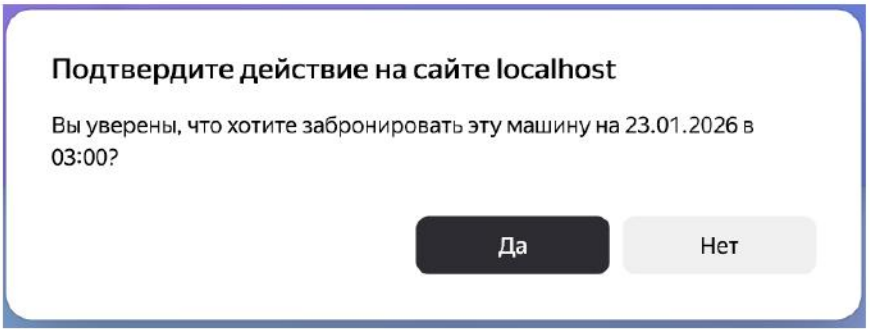


Рисунок 12 – Подтверждение бронирования

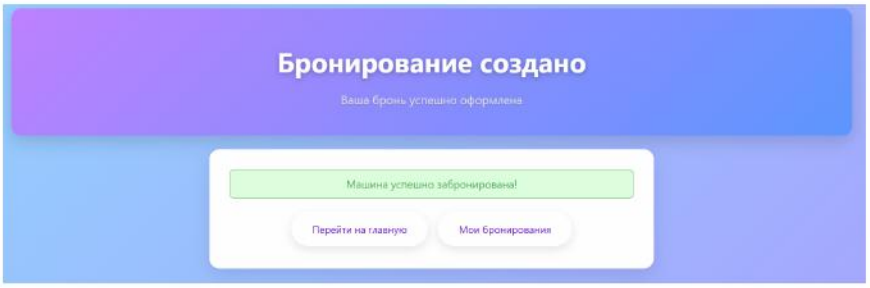


Рисунок 13 – Успешное завершение бронирования

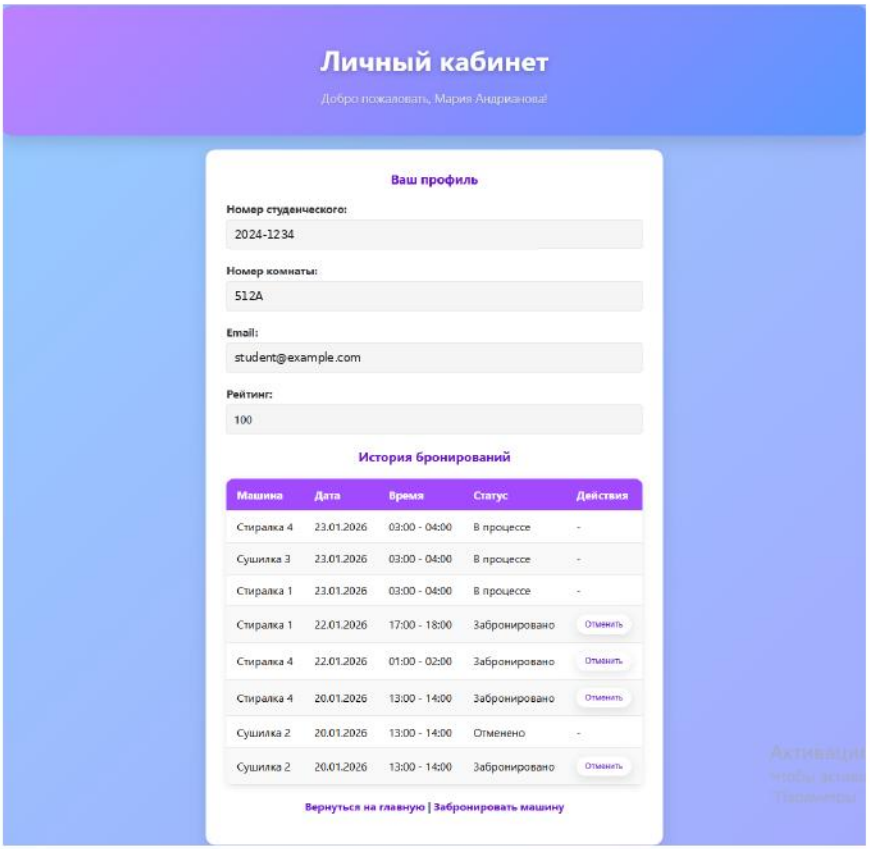


Рисунок 14 – Личный кабинет студента

При наличии неподтверждённых случаев пунктуальности система показывает уведомление. Студент подтверждает, забрал ли предыдущий пользователь вещи вовремя (рисунок 15). Если подтверждение отрицательное, рейтинг предыдущего пользователя уменьшается на 5 баллов.

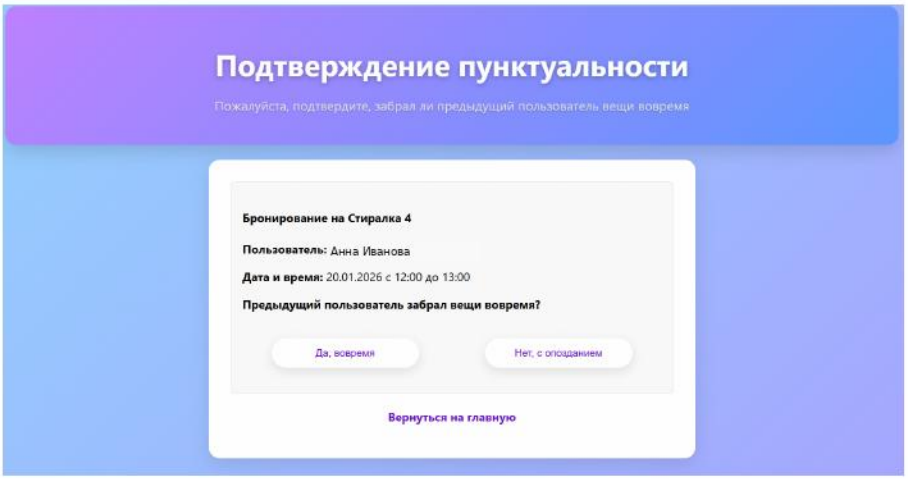


Рисунок 15 – Подтверждение пунктуальности предыдущего студента

Для сообщения о неисправностях машин студент использует страницу "Сообщить о поломке"(рисунок 16). При отправке сообщения система автоматически переводит указанную машину в статус «на ремонте» и уведомляет администратора.

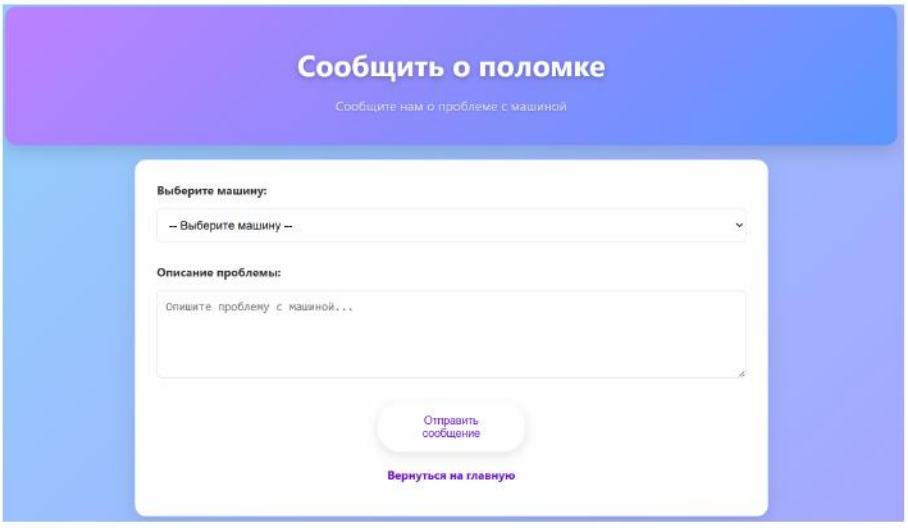


Рисунок 16 – Окно сообщения о поломке машин

На главной странице администратор имеет доступ к дополнительным разделам через навигационное меню: «Управление машинами», «История бронирований» и «Статистика» (рисунок 17).

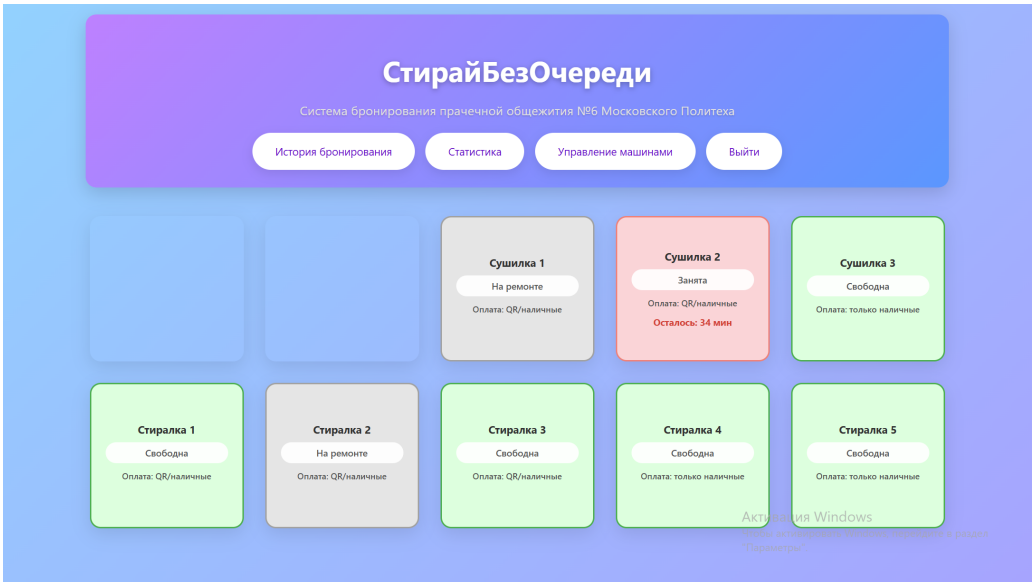


Рисунок 17 – Главная страница администратора

В разделе «Управление машинами» администратор может вручную изменять статусы любой машины. (рисунок 18). Здесь же отображаются все нерешённые сообщения о поломках — администратор может добавить заметки и отметить проблему как решённую, после чего машина автоматически возвращается в статус «свободно».

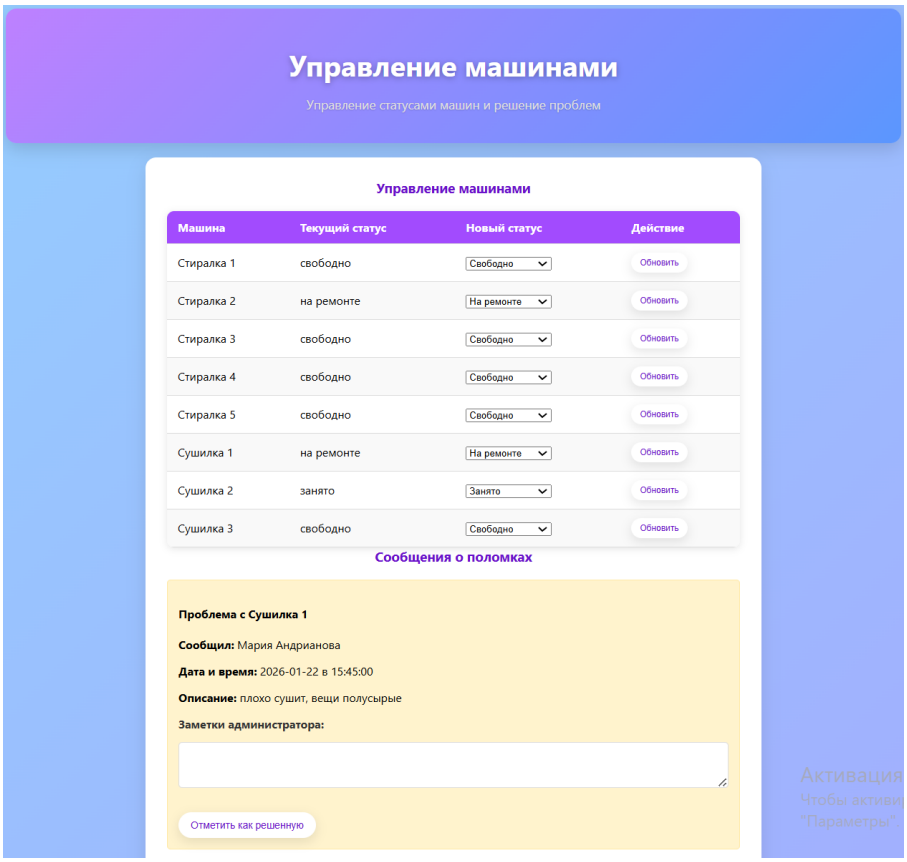


Рисунок 18 – Раздел «Управление машинами»

Раздел «История бронирований» предоставляет администратору полный доступ ко всем бронированиям с фильтрами: по дате, времени, пользователю и машине (рисунок 19).

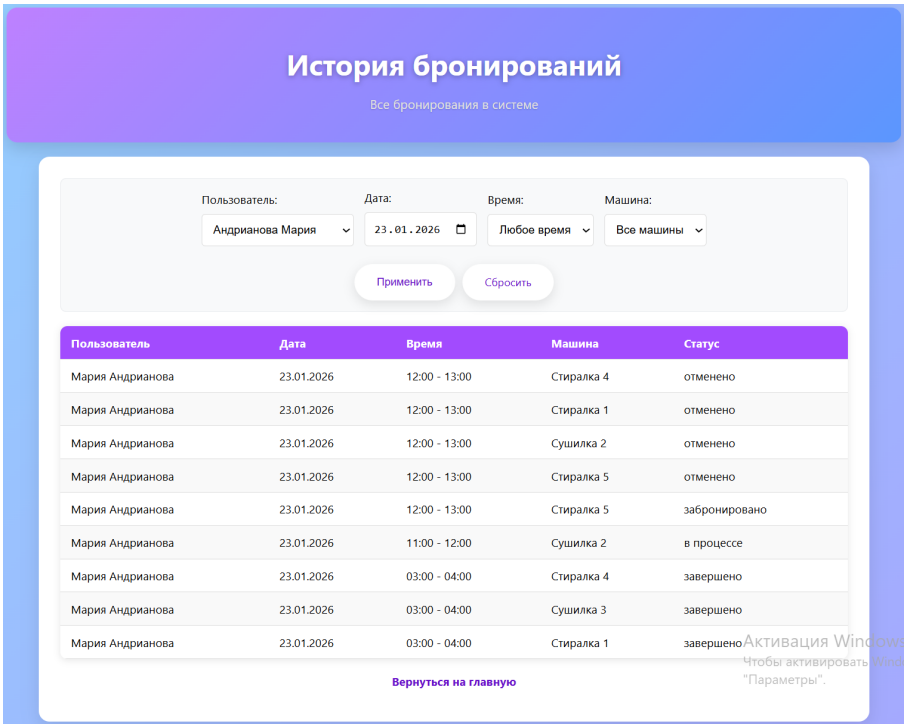


Рисунок 19 – Раздел «История бронирований»

Раздел «Статистика» отображает визуальную диаграмму загруженности прачечной (рисунок 20). Она показывает распределение бронирований по дням недели и часам.

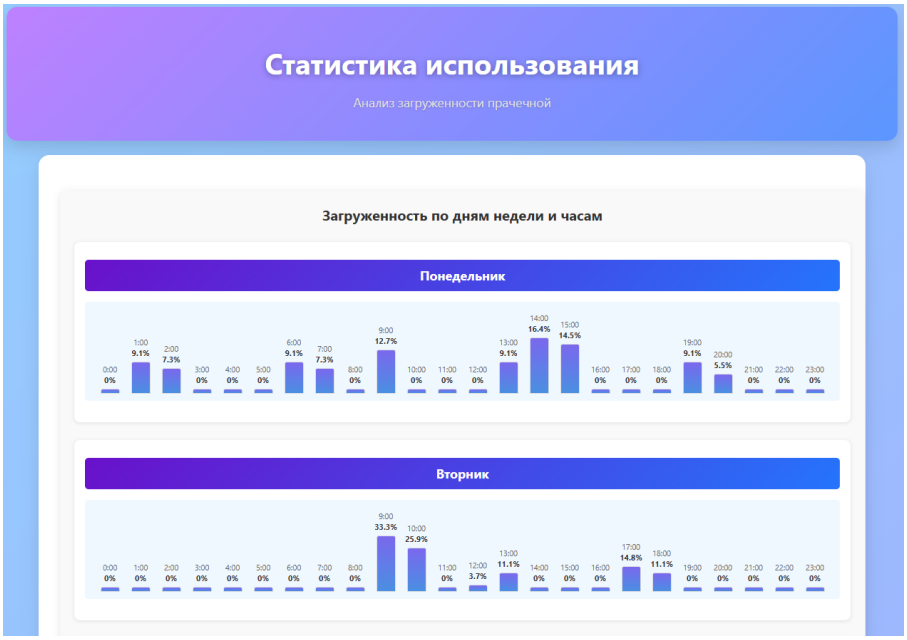


Рисунок 20 – Раздел «Статистика»

Заключение

В результате выполнения курсового проекта был разработан и реализован специализированный веб-сервис «СтирайБезОчереди» для автоматизации доступа к ресурсам прачечной студенческого общежития №6 Московского Политеха. Система представляет собой полнофункциональное клиент-серверное веб-приложение, построенное на технологическом стеке PHP и MySQL, с адаптивным пользовательским интерфейсом на HTML, CSS и JavaScript.

Ключевыми достижениями проекта являются: успешная реализация системы онлайн-бронирования с визуализацией статуса оборудования в реальном времени; внедрение механизма социальной ответственности через систему репутации пользователей; создание административной панели для мониторинга и управления ресурсами; обеспечение безопасности данных через параметризованные SQL-запросы и хэширование паролей.

Основными преимуществами разработанного решения являются: полное устранение проблемы живых очередей в прачечной; интуитивно понятный интерфейс с интерактивной картой оборудования; адаптивный дизайн, работающий на компьютерах и мобильных устройствах; автоматическое поддержание актуальности статусов машин; мотивационная система рейтинга, стимулирующая соблюдение временных рамок.

Среди ограничений системы можно отметить: зависимость от честности пользователей при подтверждении пунктуальности; необходимость ручной инициации процесса бронирования (отсутствие напоминаний); базовый визуальный дизайн интерфейса.

В дальнейшем проект может быть расширен за счёт: интеграции с системой уведомлений (Telegram-бот или email-рассылка); разработки мобильного приложения; масштабирования на все общежития университета с внедрением в личный кабинет студента.

Ссылки на проект:

- Веб-приложение: `http://f1221429.xsph.ru`
- Репозиторий с исходным кодом: `https://github.com/Maria-Git2025/CourseProject_3_semester`

Список литературы и интернет-ресурсов

[1] Курс «Проектирование баз данных» в СДО Московского Политеха [Электронный ресурс]. URL: https://lms.mospolytech.ru/course/view.php?id=15767 (дата обращения: 20.12.2025).

[2] PHP: Официальная документация [Электронный ресурс]. URL: https://www.php.net/manual/ru/ (дата обращения: 15.01.2026).

[3] Sprinthost: Документация хостинг-провайдера [Электронный ресурс]. URL: https://sprinthost.ru/docs/ (дата обращения: 22.01.2026).

[4] LaTeX Overleaf: Онлайн-редактор LaTeX [Электронный ресурс]. URL: https://www.overleaf.com/ (дата обращения: 21.01.2026).

Приложения

Приложение А – SQL-скрипт создания таблиц базы данных системы

```
CREATE TABLE users (
    id INT PRIMARY KEY AUTO_INCREMENT,
    student_card_number VARCHAR(50) NOT NULL UNIQUE,
    room_number VARCHAR(20) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    first_name VARCHAR(100) NOT NULL,
    middle_name VARCHAR(100),
    email VARCHAR(255) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    rating INT DEFAULT 100,
    is_admin BOOLEAN NOT NULL DEFAULT 0
);

CREATE TABLE machines (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL UNIQUE,
    type ENUM('стиральная', 'сушильная') NOT NULL,
    status ENUM('свободно', 'занято', 'на ремонте') DEFAULT 'свободно',
    payment_method ENUM('QR/наличные', 'только наличные') NOT NULL,
    position_x ENUM('1', '2', '3', '4', '5') NOT NULL,
    position_y ENUM('1', '2') NOT NULL,
    UNIQUE KEY unique_position (position_x, position_y)
);

CREATE TABLE bookings (
    id INT PRIMARY KEY AUTO_INCREMENT,
    id_user INT NOT NULL,
    id_machine INT NOT NULL,
    date DATE NOT NULL,
    start_time TIME NOT NULL,
    end_time TIME NOT NULL,
    status ENUM('забронировано', 'в процессе', 'завершено', 'отменено')
        DEFAULT 'забронировано',
    id_next_user INT,
    was_on_time BOOLEAN,
    UNIQUE KEY unique_machine_start (id_machine, date, start_time),
    UNIQUE KEY unique_machine_end (id_machine, date, end_time),

    FOREIGN KEY (id_user) REFERENCES users(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    FOREIGN KEY (id_machine) REFERENCES machines(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
```

```
    FOREIGN KEY (id_next_user) REFERENCES users(id)
        ON DELETE SET NULL
        ON UPDATE CASCADE
);

CREATE TABLE machine_issues (
    id INT PRIMARY KEY AUTO_INCREMENT,
    id_machine INT NOT NULL,
    reported_by INT,
    description TEXT NOT NULL,
    status ENUM('новое', 'решается', 'решено') DEFAULT 'новое',
    reported_date DATE NOT NULL,
    reported_time TIME NOT NULL,
    resolved_date DATE,
    resolved_time TIME,
    admin_notes TEXT,
    UNIQUE KEY unique_machine_report_time (id_machine, reported_date,
    reported_time),

    FOREIGN KEY (id_machine) REFERENCES machines(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    FOREIGN KEY (reported_by) REFERENCES users(id)
        ON DELETE SET NULL
        ON UPDATE CASCADE
);
```

Приложение Б – Расположение машин в реальной жизни

